

SGT UNIVERSITY

School of Engineering and Technology

Department of CSE

DATABASE MANAGEMENT SYSTEMS LABORATORY

Experiment 4

SELECT Queries — DISTINCT, WHERE, BETWEEN, IN, ORDER BY

Field	Details
Student	Harshit Khemani
Roll No.	241302081
Programme	B.Tech CSE (AI/ML)
Section	Section - C
Lab Date	09 September 2026
Dataset	employee — 10 sample rows × 8 attributes
Methods	SELECT, DISTINCT, WHERE, BETWEEN, IN, ORDER BY

1. Aim

To create an employee table, insert ten sample records, and retrieve data with SELECT using DISTINCT, WHERE (comparison, OR, BETWEEN, IN / NOT IN), and ORDER BY in ascending and descending order.

2. Theory

2.1 SELECT

SELECT reads rows from a relation. SELECT * returns every column; naming columns returns only those attributes. A query without a WHERE clause returns every row that currently exists in the table.

2.2 DISTINCT

SELECT DISTINCT removes duplicate values from the result. SELECT department_no lists a department once per employee; SELECT DISTINCT department_no lists each department number only once. Use DISTINCT when the question is about unique values, not about every row.

2.3 WHERE

The WHERE clause keeps rows that satisfy a condition. Comparison operators such as > and <> filter numbers and strings. OR keeps a row if either predicate is true (city is Delhi or Mumbai). City values in this lab are stored in title case and matched with the same casing so the filters work in SQLite without extra case-conversion functions.

2.4 BETWEEN

BETWEEN low AND high is an inclusive range: the endpoints are part of the result. NOT BETWEEN is the complement — salaries strictly below 30000 or strictly above 50000. It is equivalent to salary < 30000 OR salary > 50000 when the range is 30000 to 50000.

2.5 IN

IN (...) tests membership in a list. city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore') keeps those four cities. NOT IN keeps every city that is not in the list — here Bangalore, Hyderabad, and Pune remain when Mumbai, Delhi, and Chennai are excluded.

2.6 ORDER BY

ORDER BY sorts the result. ASC (the default) lists names A→Z; DESC lists them Z→A. Sorting does not change stored rows; it only changes the order of the result set.

3. Schema

One table, employee, holds eight attributes. sr_no is the employee number (primary key). manager stores the manager's name, or NULL for the top manager.

Attribute	Type	Constraint	Role
sr_no	INTEGER	PRIMARY KEY	Employee number
employee_name	TEXT	NOT NULL	Employee name
job	TEXT	NOT NULL	Job title
manager	TEXT	NULL allowed	Manager name
hire_date	DATE	NOT NULL	Date of joining
salary	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary
department_no	INTEGER	NOT NULL	Department number
city	TEXT	NOT NULL	Work city

4. Procedure

1. Drop employee if it already exists, so the script can be re-run.
2. Create the table with the eight attributes listed above.
3. Insert ten employees with mixed salaries, cities, and department numbers.
4. Select all columns from employee.
5. Select distinct department numbers, then all department numbers without DISTINCT.
6. Filter by salary (> 30000, BETWEEN, NOT BETWEEN).
7. Filter by city (not Delhi, Delhi or Mumbai, IN, NOT IN).
8. Sort the full table by employee name descending, then ascending.

5. Source Code

SQLite script used in the lab compiler (09-09-2026/employee.sql). For the class SQL Server, run employee.sqlserver.sql.

```
-- =====
-- DBMS Lab · 09-09-2026
-- Employee table: CREATE, seed 10 rows, then SELECT queries
-- DISTINCT, WHERE, BETWEEN, IN, ORDER BY
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employee.sqlserver.sql instead.
-- =====

DROP TABLE IF EXISTS employee;

-- -----
-- 1. Create the employee table
-- -----
CREATE TABLE employee (
  sr_no          INTEGER PRIMARY KEY,
  employee_name  TEXT NOT NULL,
  job            TEXT NOT NULL,
  manager        TEXT,
  hire_date      DATE NOT NULL,
  salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
  department_no  INTEGER NOT NULL,
  city           TEXT NOT NULL
);

-- -----
-- 2. Insert 10 sample rows
--   · salaries below, inside, and above 30000-50000
--   · cities: Delhi, Mumbai, Chennai, Bangalore, Hyderabad, Pune
--   · mixed department numbers (some duplicates)
--   · Rajesh Kumar is the top manager (manager IS NULL)
-- -----
INSERT INTO employee
(sr_no, employee_name, job, manager, hire_date, salary, department_no, city)
```

```
VALUES
(1, 'Rajesh Kumar', 'General Manager', NULL, '2018-03-12', 85000.00, 10, 'Delhi'),
(2, 'Ananya Sharma', 'Software Engineer', 'Rajesh Kumar', '2021-07-01', 45000.00, 20, 'Bangalore'),
(3, 'Rohan Mehta', 'Accountant', 'Rajesh Kumar', '2020-01-15', 28000.00, 30, 'Mumbai'),
(4, 'Priya Nair', 'HR Executive', 'Rajesh Kumar', '2022-04-20', 32000.00, 40, 'Chennai'),
(5, 'Vikram Singh', 'Sales Manager', 'Rajesh Kumar', '2019-11-08', 52000.00, 50, 'Hyderabad'),
(6, 'Fatima Khan', 'Software Engineer', 'Ananya Sharma', '2023-02-14', 38000.00, 20, 'Bangalore'),
(7, 'Arjun Patel', 'Clerk', 'Rohan Mehta', '2024-06-01', 18000.00, 30, 'Pune'),
(8, 'Neha Gupta', 'Marketing Analyst', 'Rajesh Kumar', '2021-09-10', 35000.00, 60, 'Delhi'),
(9, 'Karan Iyer', 'Database Admin', 'Ananya Sharma', '2020-12-05', 48000.00, 20, 'Mumbai'),
(10, 'Meera Joshi', 'Receptionist', 'Priya Nair', '2023-08-22', 22000.00, 40, 'Chennai');
```

```
-- 3. Select all details from employee
```

```
-- All employee rows
SELECT * FROM employee;
```

```
-- 4. Select only DISTINCT department numbers
```

```
-- Distinct department numbers
SELECT DISTINCT department_no FROM employee;
```

```
-- 5. Select all department numbers (no DISTINCT)
```

```
-- All department numbers
SELECT department_no FROM employee;
```

```
-- 6. Employee name and salary where salary > 30000
```

```
-- Salary greater than 30000
SELECT employee_name, salary
FROM employee
WHERE salary > 30000;
```

```
-- 7. Employee name and hire date where city is NOT Delhi
```

```
-- City is not Delhi
SELECT employee_name, hire_date
FROM employee
WHERE city <> 'Delhi';
```

```
-- 8. Employee name and hire date where city is Delhi OR Mumbai
```

```
-- City is Delhi or Mumbai
SELECT employee_name, hire_date
FROM employee
WHERE city = 'Delhi' OR city = 'Mumbai';
```

```
-- 9. Employee number and name where salary BETWEEN 30000 AND 50000
```

```
-- Salary between 30000 and 50000
SELECT sr_no, employee_name
FROM employee
WHERE salary BETWEEN 30000 AND 50000;
```

```
-- 10. Employee number and name where salary NOT BETWEEN 30000 AND 50000
```

```
-- Salary not between 30000 and 50000
SELECT sr_no, employee_name
FROM employee
WHERE salary NOT BETWEEN 30000 AND 50000;
```

```
-- 11. All details where city IN (Delhi, Mumbai, Chennai, Bangalore)
```

```
-- City in metro list
SELECT *
FROM employee
WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');
```

```
-- 12. All details where city NOT IN (Mumbai, Delhi, Chennai)
```

```
-- City not in Mumbai, Delhi, Chennai
SELECT *
FROM employee
WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');
```

```
-- 13. All details ordered by employee name DESC
```

```
-- Employees in descending name order
```

```

SELECT *
FROM employee
ORDER BY employee_name DESC;

-----
-- 14. All details ordered by employee name ASC
-----
-- Employees in ascending name order
SELECT *
FROM employee
ORDER BY employee_name ASC;

```

6. Output

The script was executed in SQLite in memory. CREATE and INSERT print a status line; each SELECT prints its result set. NULL in the manager column is shown as an empty cell.

```

SQL> DROP TABLE IF EXISTS employee;
-- OK

SQL> CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,
    job            TEXT NOT NULL,
    manager        TEXT,
    hire_date      DATE NOT NULL,
    salary          NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    department_no  INTEGER NOT NULL,
    city           TEXT NOT NULL
);
-- OK

SQL> INSERT INTO employee
(sr_no, employee_name, job, manager, hire_date, salary, department_no, city)
VALUES
(1, 'Rajesh Kumar', 'General Manager', NULL, '2018-03-12', 85000.00,
10, 'Delhi'),
(2, 'Ananya Sharma', 'Software Engineer', 'Rajesh Kumar', '2021-07-01', 45000.00,
20, 'Bangalore'),
(3, 'Rohan Mehta', 'Accountant', 'Rajesh Kumar', '2020-01-15', 28000.00,
30, 'Mumbai'),
(4, 'Priya Nair', 'HR Executive', 'Rajesh Kumar', '2022-04-20', 32000.00,
40, 'Chennai'),
(5, 'Vikram Singh', 'Sales Manager', 'Rajesh Kumar', '2019-11-08', 52000.00,
50, 'Hyderabad'),
(6, 'Fatima Khan', 'Software Engineer', 'Ananya Sharma', '2023-02-14', 38000.00,
20, 'Bangalore'),
(7, 'Arjun Patel', 'Clerk', 'Rohan Mehta', '2024-06-01', 18000.00,
30, 'Pune'),
(8, 'Neha Gupta', 'Marketing Analyst', 'Rajesh Kumar', '2021-09-10', 35000.00,
60, 'Delhi'),
(9, 'Karan Iyer', 'Database Admin', 'Ananya Sharma', '2020-12-05', 48000.00,
20, 'Mumbai'),
(10, 'Meera Joshi', 'Receptionist', 'Priya Nair', '2023-08-22', 22000.00,
40, 'Chennai');
-- 10 row(s) inserted

SQL> SELECT * FROM employee;
sr_no  employee_name  job            manager      hire_date  salary  department_no  city
-----
1      Rajesh Kumar   General Manager                   2018-03-12  85000   10             Delhi
2      Ananya Sharma  Software Engineer    Rajesh Kumar 2021-07-01  45000   20             Bangalore
3      Rohan Mehta    Accountant           Rajesh Kumar 2020-01-15  28000   30             Mumbai
4      Priya Nair     HR Executive         Rajesh Kumar 2022-04-20  32000   40             Chennai
5      Vikram Singh   Sales Manager        Rajesh Kumar 2019-11-08  52000   50             Hyderabad
6      Fatima Khan    Software Engineer    Ananya Sharma 2023-02-14  38000   20             Bangalore
7      Arjun Patel    Clerk                Rohan Mehta  2024-06-01  18000   30             Pune
8      Neha Gupta     Marketing Analyst     Rajesh Kumar 2021-09-10  35000   60             Delhi
9      Karan Iyer     Database Admin        Ananya Sharma 2020-12-05  48000   20             Mumbai
10     Meera Joshi    Receptionist         Priya Nair   2023-08-22  22000   40             Chennai

SQL> SELECT DISTINCT department_no FROM employee;
department_no
-----
10
20
30
40
50
60

SQL> SELECT department_no FROM employee;
department_no
-----
10
20
30
40
50
20
30
60
20
40

SQL> SELECT employee_name, salary

```

```

FROM employee
WHERE salary > 30000;
employee_name salary
-----
Rajesh Kumar 85000
Ananya Sharma 45000
Priya Nair 32000
Vikram Singh 52000
Fatima Khan 38000
Neha Gupta 35000
Karan Iyer 48000

```

```

SQL> SELECT employee_name, hire_date
FROM employee
WHERE city <> 'Delhi';
employee_name hire_date
-----
Ananya Sharma 2021-07-01
Rohan Mehta 2020-01-15
Priya Nair 2022-04-20
Vikram Singh 2019-11-08
Fatima Khan 2023-02-14
Arjun Patel 2024-06-01
Karan Iyer 2020-12-05
Meera Joshi 2023-08-22

```

```

SQL> SELECT employee_name, hire_date
FROM employee
WHERE city = 'Delhi' OR city = 'Mumbai';
employee_name hire_date
-----
Rajesh Kumar 2018-03-12
Rohan Mehta 2020-01-15
Neha Gupta 2021-09-10
Karan Iyer 2020-12-05

```

```

SQL> SELECT sr_no, employee_name
FROM employee
WHERE salary BETWEEN 30000 AND 50000;
sr_no employee_name
-----
2 Ananya Sharma
4 Priya Nair
6 Fatima Khan
8 Neha Gupta
9 Karan Iyer

```

```

SQL> SELECT sr_no, employee_name
FROM employee
WHERE salary NOT BETWEEN 30000 AND 50000;
sr_no employee_name
-----
1 Rajesh Kumar
3 Rohan Mehta
5 Vikram Singh
7 Arjun Patel
10 Meera Joshi

```

```

SQL> SELECT *
FROM employee
WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');
sr_no employee_name job manager hire_date salary department_no city
-----
1 Rajesh Kumar General Manager 2018-03-12 85000 10 Delhi
2 Ananya Sharma Software Engineer Rajesh Kumar 2021-07-01 45000 20 Bangalore
3 Rohan Mehta Accountant Rajesh Kumar 2020-01-15 28000 30 Mumbai
4 Priya Nair HR Executive Rajesh Kumar 2022-04-20 32000 40 Chennai
6 Fatima Khan Software Engineer Ananya Sharma 2023-02-14 38000 20 Bangalore
8 Neha Gupta Marketing Analyst Rajesh Kumar 2021-09-10 35000 60 Delhi
9 Karan Iyer Database Admin Ananya Sharma 2020-12-05 48000 20 Mumbai
10 Meera Joshi Receptionist Priya Nair 2023-08-22 22000 40 Chennai

```

```

SQL> SELECT *
FROM employee
WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');
sr_no employee_name job manager hire_date salary department_no city
-----
2 Ananya Sharma Software Engineer Rajesh Kumar 2021-07-01 45000 20 Bangalore
5 Vikram Singh Sales Manager Rajesh Kumar 2019-11-08 52000 50 Hyderabad
6 Fatima Khan Software Engineer Ananya Sharma 2023-02-14 38000 20 Bangalore
7 Arjun Patel Clerk Rohan Mehta 2024-06-01 18000 30 Pune

```

```

SQL> SELECT *
FROM employee
ORDER BY employee_name DESC;
sr_no employee_name job manager hire_date salary department_no city
-----
5 Vikram Singh Sales Manager Rajesh Kumar 2019-11-08 52000 50 Hyderabad
3 Rohan Mehta Accountant Rajesh Kumar 2020-01-15 28000 30 Mumbai
1 Rajesh Kumar General Manager 2018-03-12 85000 10 Delhi
4 Priya Nair HR Executive Rajesh Kumar 2022-04-20 32000 40 Chennai
8 Neha Gupta Marketing Analyst Rajesh Kumar 2021-09-10 35000 60 Delhi
10 Meera Joshi Receptionist Priya Nair 2023-08-22 22000 40 Chennai
9 Karan Iyer Database Admin Ananya Sharma 2020-12-05 48000 20 Mumbai
6 Fatima Khan Software Engineer Ananya Sharma 2023-02-14 38000 20 Bangalore
7 Arjun Patel Clerk Rohan Mehta 2024-06-01 18000 30 Pune
2 Ananya Sharma Software Engineer Rajesh Kumar 2021-07-01 45000 20 Bangalore

```

```

SQL> SELECT *
FROM employee
ORDER BY employee_name ASC;
sr_no employee_name job manager hire_date salary department_no city
-----

```

2	Ananya Sharma	Software Engineer	Rajesh Kumar	2021-07-01	45000	20	Bangalore
7	Arjun Patel	Clerk	Rohan Mehta	2024-06-01	18000	30	Pune
6	Fatima Khan	Software Engineer	Ananya Sharma	2023-02-14	38000	20	Bangalore
9	Karan Iyer	Database Admin	Ananya Sharma	2020-12-05	48000	20	Mumbai
10	Meera Joshi	Receptionist	Priya Nair	2023-08-22	22000	40	Chennai
8	Neha Gupta	Marketing Analyst	Rajesh Kumar	2021-09-10	35000	60	Delhi
4	Priya Nair	HR Executive	Rajesh Kumar	2022-04-20	32000	40	Chennai
1	Rajesh Kumar	General Manager		2018-03-12	85000	10	Delhi
3	Rohan Mehta	Accountant	Rajesh Kumar	2020-01-15	28000	30	Mumbai
5	Vikram Singh	Sales Manager	Rajesh Kumar	2019-11-08	52000	50	Hyderabad

7. Results

Ten employees load successfully. The filters return non-empty, distinct subsets as expected for this seed data:

Query	Rows	What the result shows
SELECT *	10	Every employee and every column
DISTINCT department_no	6	Departments 10, 20, 30, 40, 50, 60
department_no (no DISTINCT)	10	Duplicates kept (dept 20 appears three times)
salary > 30000	7	Excludes 18000, 22000, 28000
city <> Delhi	8	Keeps everyone except Rajesh and Neha
city Delhi OR Mumbai	4	Rajesh, Rohan, Neha, Karan
salary BETWEEN 30000 AND 50000	5	Inclusive range; excludes low and high pay
salary NOT BETWEEN 30000 AND 50000	5	Complement of the BETWEEN query
city IN (four metros)	8	Drops Hyderabad and Pune
city NOT IN (Mumbai, Delhi, Chennai)	4	Bangalore, Hyderabad, Pune remain
ORDER BY name DESC / ASC	10	Same ten rows; opposite sort orders

DISTINCT collapses ten department values to six unique numbers. BETWEEN and NOT BETWEEN partition the table with no overlap ($5 + 5 = 10$). City membership tests leave Hyderabad and Pune only in the NOT IN result, which is the check that the seed data includes cities outside the four-metro list.

8. Conclusion

A single SELECT can project columns, remove duplicates, filter with comparison and set predicates, and sort the result. DISTINCT answers “which unique values exist?” WHERE answers “which rows match this condition?” BETWEEN and IN express ranges and lists clearly. ORDER BY changes presentation only. Together these clauses are the core of everyday retrieval in SQL.